

Airgap — Project Context & Week 1 Plan

Drop this file in the root of the `querygpt` repo. At the start of every Claude Code session this week, say: "Read AIRGAP.md, we're on Day N."

1. What this is

One line: Drop a CSV into the browser. Ask questions in plain English. The data never leaves the tab.

Lineage: This is the direct upgrade of `querygpt` (NF QueryGPT). QueryGPT already proved the core loop — NL question → generated SQL → shown to the user → results as table/chart — against a fixed server-side schema for one company. Airgap keeps that entire loop and changes three things:

	QueryGPT (done)	Airgap (this week)
Where the DB lives	Server (Postgres/SQLite)	Browser (DuckDB-WASM)
What data it works on	NikahForever's fixed schema	Any CSV/Parquet the user drops in
What reaches the LLM	Question (+ server queries real data)	Question + schema only — never rows

Everything else — `ResponseCard.tsx`, `LoadingPipeline.tsx`, the SQL-always-visible principle, the conversation history — carries over.

2. Hard rules (non-negotiable all week)

- Row data never crosses the network.** Only column names/types (schema) are ever sent to `/api/query` or to Claude. This is the entire product thesis — if it slips, the project has no reason to exist.
- Generated SQL is always shown to the user**, never hidden. (QueryGPT already does this — keep it.)
- Next.js stays.** Do not migrate to Vite. All the reasoning for "no backend, no framework tax" is about runtime architecture (no server does the querying), not about which tool

builds the shell. QueryGPT's UI, fonts, and Tailwind config already work — reuse them. The one server-side thing that remains is a thin API route that calls Claude with schema-only input (see §5) — an API key can't live in client code, so that route has to exist regardless of framework.

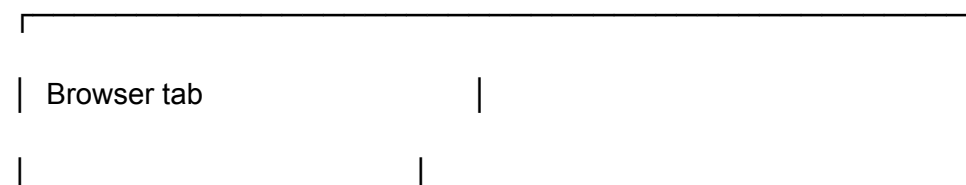
4. **Single-threaded DuckDB-WASM only, this week.** Do NOT add `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy` headers. The multi-threaded (COI) bundle needs those headers, and they're known to break other things on a site (auth popups, third-party embeds, image loading) unless you deal with it deliberately. The default single-threaded bundle DuckDB-WASM auto-selects is fully correct — just not parallelized — which is irrelevant for a single-user local query. Revisit only if a specific perf problem shows up.

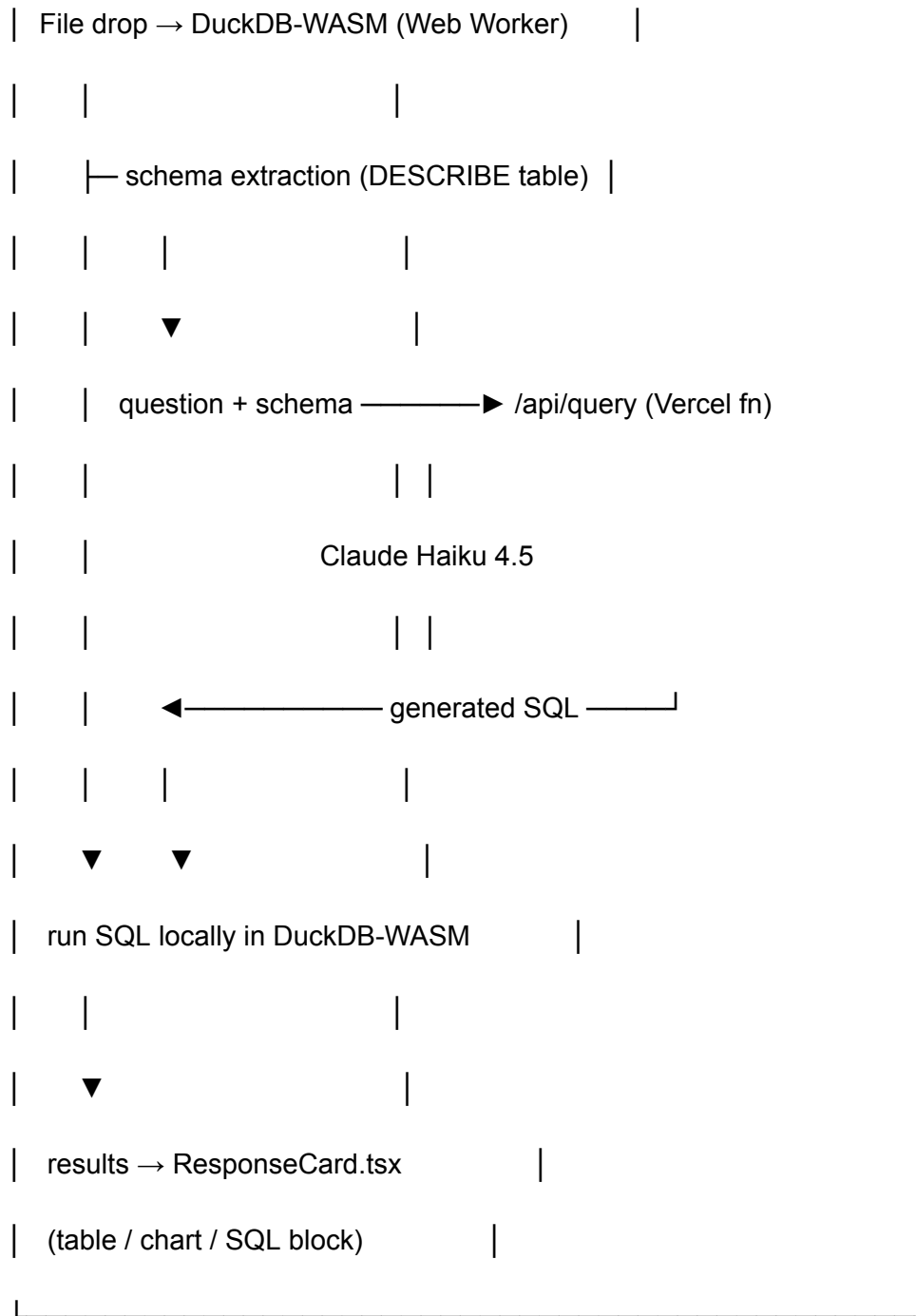
3. What already exists (reuse map)

From the current repo:

- `app/page.tsx` — UI + data flow (`ask()`, optimistic exchanges, rolling history) — **keep the shape, change the data source**
- `components/ResponseCard.tsx` — renders result states incl. SQL block, bars, tables — **generalize from fixed demo shape to dynamic columns**
- `components/LoadingPipeline.tsx` — animated loading state — **reuse as-is for query execution**
- `lib/types.ts` — defines `ApiResponse` (`result` | `clarify` | `blocked` | `error`) — **open this Day 1 and confirm exactly what shape `result` currently expects (does it include `rows/columns` directly, or something else?) before touching the API route. Everything in §5 assumes the common case; adjust to what's actually there.**
- `system_prompt.txt` — existing NL→SQL system prompt — **adapt, don't replace: change from "you know the schema" to "you only ever see schema, never data"**
- `app/api/query/route.ts` — currently a demo stub — **this becomes the schema-only Claude proxy**

4. Target architecture





Only "question + schema" ever crosses the top of that box. Rows never do.

5. API contract change

Old (QueryGPT demo stub): `POST /api/query` takes `{question, history}`, pattern-matches, returns a full `result` with fake rows already filled in.

New (Airgap): `POST /api/query` takes `{question, schema, history}` where `schema` is just `[{name, type}]` — no data. Claude Haiku returns **SQL text only**. The client then executes that SQL against its local DuckDB instance and builds the `result` shape itself before handing it to `ResponseCard`.

Net effect: the server's job shrinks (`schema` → `SQL`), and a new client-side step appears (`SQL` → rows, locally). Confirm the exact `result` fields in `lib/types.ts` on Day 1 so this slots in cleanly rather than fighting the existing types.

6. New dependencies

`npm install @duckdb/duckdb-wasm`

`npm install @tanstack/react-virtual`

CodeMirror 6 only if Day 3 goes fast — plain textarea is fine to start

7. Starting skeleton for `lib/duckdb.ts`

This is a starting point, not gospel — have Claude Code check it against the actually-installed package version before trusting it byte-for-byte (duckdb-wasm's API surface shifts slightly between releases):

```
import * as duckdb from '@duckdb/duckdb-wasm';

let dbInstance: duckdb.AsyncDuckDB | null = null;

export async function getDB() {

  if (dbInstance) return dbInstance;

  const bundles = duckdb.getJsDelivrBundles();

  const bundle = await duckdb.selectBundle(bundles); // auto-picks single-threaded MVP/EH —
  see rule #4
```

```

const workerUrl = URL.createObjectURL(
  new Blob(['importScripts("${bundle.mainWorker}");'], { type: 'text/javascript' })
);

const worker = new Worker(workerUrl);

const logger = new duckdb.ConsoleLogger();

const db = new duckdb.AsyncDuckDB(logger, worker);

await db.instantiate(bundle.mainModule, bundle.pthreadWorker);

URL.revokeObjectURL(workerUrl);

dbInstance = db;

return db;
}

export async function loadFile(file: File) {

  const db = await getDB();

  await db.registerFileHandle(file.name, file,
    duckdb.DuckDBDataProtocol.BROWSER_FILEREADER, true);

  const conn = await db.connect();

  await conn.query(`CREATE TABLE data AS SELECT * FROM read_csv_auto('${file.name}')`);

  const schema = await conn.query(`DESCRIBE data`);

  await conn.close();

  return schema; // column_name, column_type, ...
}

export async function runSQL(sql: string) {

```

```
const db = await getDB();

const conn = await db.connect();

const result = await conn.query(sql);

await conn.close();

return result;

}
```

8. Explicitly OUT of scope this week

Cutting these on purpose so Tier 1 actually ships. They're not forgotten, just not Week 1:

- OPFS persistence across tab closes
 - Multi-file joins
 - Auto-EDA report
 - Remote Parquet-over-HTTP
 - CodeMirror autocomplete (plain textarea is fine)
 - The  byte-counter badge can be a static claim this week ("schema-only, verify in your Network tab") rather than a live-computed counter — make it real in week 2 if this continues.
-

9. Day-by-day (7 days)

Day 1 — DuckDB core, no UI

- `npm install @duckdb/duckdb-wasm`, add `lib/duckdb.ts` from §7
- Open `lib/types.ts`, confirm the real `result` shape
- Hardcode a small CSV, run `SELECT * LIMIT 10`, log to console
- **Done when:** a real CSV's rows print in the browser console, zero network calls involved

Day 2 — Drag-drop + schema

- Drag-drop file input (adapt existing input area in `app/page.tsx`)
- On drop → `loadFile()` → store schema in React state

- Build a simple schema sidebar: column name, type
- **Done when:** drop a real CSV, see an accurate schema list on screen

Day 3 — Grid + manual SQL

- Plain `<textarea>` for raw SQL input
- Wire `runSQL()` results into `ResponseCard.tsx`'s table path (generalize it from the fixed demo shape to dynamic columns)
- Add `@tanstack/react-virtual` to the results table
- **Done when:** typing SQL by hand renders a virtualized table via the existing UI

Day 4 — NL→SQL rewire (the actual upgrade)

- Rewrite `app/api/query/route.ts`: strip the demo stub, call Claude Haiku 4.5 with `{question, schema, history}` only
- Adapt `system_prompt.txt`: "you only ever see schema, never rows; output DuckDB SQL only"
- Client: submit question → get SQL back → `runSQL()` locally → shape into `result` → feed to `ResponseCard`
- **Done when:** ask a plain-English question, get correct results + visible SQL, and the Network tab shows only schema was ever sent

Day 5 — Charts + the privacy claim visible in the UI

- Reuse/extend the existing bar chart in `ResponseCard` for numeric aggregations
- Add the static "🔒 schema-only, verify in Network tab" badge
- **Done when:** ask → answer → SQL → chart, feels like one solid loop

Day 6 — Stress test + error states

- Try a 500MB–1GB CSV, confirm the Worker keeps the UI responsive (loading state via `LoadingPipeline.tsx`)
- Bad file, malformed SQL, empty results — reuse existing error states in `ResponseCard`
- **Done when:** it survives a big real file and a few deliberately broken inputs without crashing

Day 7 — Ship

- Update `README.md`: what it is, the privacy pitch, a screenshot/gif
- Record the demo clip: load a large file → ask a question → get an answer → **turn off wifi** → run a manual SQL query → still works
- Deploy (Vercel — Next.js already fits; the `/api/query` route runs as a serverless function, the DB stays fully client-side)

- Post the launch thread on X
 - **Done when:** there's a live URL and a public post
-

10. How to use this file

Paste this whole document as the first message of each Claude Code session this week, followed by which day you're on. Claude Code has no memory between sessions — this file is the memory.